

AB Power Meter Monitor — Deployment & Integration Guide

Allen-Bradley (Rockwell 1403) site power-meter poller, data logger, and Prometheus exporter for NRAO / Green Bank Observatory site infrastructure monitoring.

The toolset polls AB power meters over HTTP, parses the 11 HTML table pages, accumulates a columnar time-series in memory, and writes any combination of FITS, CSV, XLSX, Markdown-log, and Veusz outputs — or serves the live values to Prometheus.

Contents

- [Components & versions](#)
- [Deployment modes](#)
- [Requirements & environment setup](#)
- [Configuration switches](#)
- [Mode A — headless monitor under systemd](#)
- [Mode B — Prometheus exporter](#)
- [Mode C — driving the monitor from Python \(the caller\)](#)
- [Graceful stop](#)
- [Disabling RAM dumps](#)
- [Python 3.7.11 compatibility](#)
- [Restart behaviour & log access](#)
- [Uninstall](#)
- [Troubleshooting](#)
- [Change log](#)

Components & versions

File	Version	Role
ab_power_meter_monitor.py	1.5.5	Main module — poller, accumulator, all output writers, optional GUI, CLI front-end
ab_meter_caller.py	1.2.10	Optional wrapper — drive the monitor one poll at a time from Python
ab_caller_Example.py	1.1.2	Minimal usage examples for the caller
ab_ramflush_off_Example.py	1.0.1	Example: caller + Prometheus with RAM dumps disabled
ab_prometheus_exporter.py	0.0.3	Prometheus exporter (uses the monitor as a library)
ab_stop.py	1.0.0	Graceful-stop helper (writes the <code>STOP_COLLECTION</code> file)
install.sh	1.0.0	systemd installer (monitor and/or exporter; system or user; venv/mamba)
abmeter_wrapper.sh	1.0.0	Single-instance wrapper for the monitor service (flock + PID)
abmeter.service	—	systemd unit template — headless monitor (Mode A)
ab-exporter.service	—	systemd unit template — Prometheus exporter (Mode B)
abmeter_logrotate.conf	—	logrotate policy (system mode, monitor logs)
install-mamba.sh	1.0.0	Pure conda-forge installer (no pip)
requirements.txt	—	pip dependency pins for Python 3.8+
requirements-py37.txt	—	pip dependency pins for Python 3.7.x
environment.yml	—	Mamba env (Python 3.8+, pip-installed deps)
environment-py37.yml	—	Mamba env (Python 3.7.11, pip-installed deps)
environment-conda.yml	—	Mamba env (Python 3.11, pure conda-forge , no pip)

The monitor accepts CLI overrides so it can be launched directly as a service without editing the file:

```
python ab_power_meter_monitor.py --headless --count 0 --interval 30 \
--ips 10.16.130.50,10.16.130.54 [--no-ram-flush] [--ram-pct 80]
```

With no flags it behaves exactly as before (reads the module-level switches).

Deployment modes

There are three ways to run the toolset. Pick one:

Mode	Use when	Entry point
A. Headless monitor	You want files (FITS/CSV/XLSX/Veusz) written on a schedule under systemd	<code>ab_power_meter_monitor.py --headless</code>
B. Prometheus exporter	You want live metrics scraped by Prometheus/Grafana	<code>ab_prometheus_exporter.py</code>
C. Python caller	You want to drive polling from your own Python code	<code>import ab_meter_caller</code>

Modes A and B are independent long-lived services and can run side by side.

Requirements & environment setup

The application code is written to the **Python 3.7 language level** and runs unmodified on 3.7.11 through 3.12+. Only the *dependency versions* differ between interpreters (see [compatibility](#)).

Preferred: mamba (interpreter) + pip (packages)

This hybrid is the recommended way to combine conda-family tooling with pip: mamba owns the interpreter, pip installs the dependencies from the pinned requirements file. It avoids conda-forge/PyPI name and version mismatches and sidesteps conda-forge’s thin Python-3.7 coverage.

```
# Python 3.8+
mamba env create -f environment.yml           # creates env "abmeter"
mamba activate abmeter

# Python 3.7.11
mamba env create -f environment-py37.yml      # creates env "abmeter37"
mamba activate abmeter37
```

Equivalent explicit form:

```
mamba create -n abmeter python=3.11
mamba run -n abmeter pip install -r requirements.txt
```

Do **not** use `mamba install --file requirements.txt`. Those files are pip-format and resolve against PyPI; conda resolves against conda-forge, where some names differ (`PySide6` → `pyside6`), the exact pins may be absent, and Python-3.7 builds are largely gone. Always let pip install the packages inside the mamba env.

Pure mamba (no pip at all)

If you want every dependency from conda-forge with no pip step, use the pure env file or the helper script. This targets modern Python (3.11) — conda-forge has largely dropped Python 3.7 builds, so a no-pip install is 3.8+ only.

```
mamba env create -f environment-conda.yml      # every dep from conda-forge
mamba activate abmeter

# or, scripted (full stack, or ABMETER_MINIMAL=1 for exporter/headless only):
./install-mamba.sh
ABMETER_MINIMAL=1 ./install-mamba.sh
```

conda-forge names differ from PyPI in a few places — the env file handles them: `PySide6` → `pyside6`, `matplotlib` → `matplotlib-base`; `prometheus_client` keeps its underscore.

Plain pip / venv

```
python3 -m venv venv && . venv/bin/activate
pip install -r requirements.txt           # 3.8+
# or, on Python 3.7.x:
pip install -r requirements-py37.txt
```

Minimal install (exporter only)

If you only run the Prometheus exporter you don’t need the FITS/GUI stack:

```
pip install "prometheus_client>=0.19" requests beautifulsoup4 lxml # 3.8+
pip install "prometheus_client==0.17.1" requests beautifulsoup4 lxml # 3.7.x
```

Configuration switches

All runtime behaviour is controlled by module-level constants near the top of `ab_power_meter_monitor.py` (the `ENABLE_*` block). A caller or host may override any of them by assignment before invoking the module, e.g. `abm.ENABLE_RAM_FLUSH = 0`.

Switch	Default	Meaning
ENABLE_GUI	1	Show the PySide6 window. Set 0 for headless / service / exporter use.
ENABLE_FITS	0	Write NRAO-compliant FITS files.
ENABLE_CSV	0	Write per-table CSV files.
ENABLE_XLSX	0	Write an Excel workbook with charts.
ENABLE_LOG_APPEND	0	Write per-device Markdown log tables.
ENABLE_LOG_FILE	0	Write <code>ab_monitor.log</code> (Python logging file handler).
ENABLE_VEUSZ	0	Write Veusz <code>.vszh5</code> project files.
ENABLE_SIGNAL_HANDLERS	1	Install SIGINT/SIGTERM handlers that convert Ctrl-C / kill into a graceful stop. Set 0 when embedding the module in a host that owns its own signals (e.g. a Prometheus exporter) — otherwise the host's SIGTERM is swallowed and the host is torn down. (v1.5.1+)
ENABLE_RAM_FLUSH	1	Master enable for automatic RAM/size flushing. Set 0 to never auto-flush or clear the store mid-run — you get one uninterrupted, fully-accumulated time-series (watch RAM on long runs). End-of-run writes and the GUI “Save Now” button are unaffected. (v1.5.2+)
IP_LIST	[...]	Explicit list of device IPs to poll.
SAMPLE_PERIOD_SEC	30	Seconds between polls.
MEM_RAM_PCT_LIMIT	60	RAM % at which an automatic flush+clear fires (only when <code>ENABLE_RAM_FLUSH=1</code>).

The two switches added for service/exporter integration are `ENABLE_SIGNAL_HANDLERS` and `ENABLE_RAM_FLUSH`; both default to the original behaviour so existing deployments are unchanged.

Mode A — headless monitor under systemd

The monitor runs in **headless mode** as a long-lived service (`HEADLESS_LOOP_COUNT = 0`, infinite loop). The `ab_meter_caller.py` wrapper is **not needed** for the service — `ab_power_meter_monitor.py` is invoked directly with `--headless --count 0`.

Do I need sudo?

Mode	Requires sudo?	Notes
System-wide (<code>sudo ./install.sh</code>)	Yes	Creates <code>/etc/systemd/system/abmeter.service</code> , an <code>abmeter</code> system user, <code>/run/abmeter/</code> , and <code>/etc/logrotate.d/abmeter</code> . Starts at boot for all users.
Per-user (<code>./install.sh --user</code>)	No	Installs to <code>~/.config/systemd/user/</code> . Starts on login (or at boot if linger is enabled).

Recommendation: dedicated monitoring host → system mode with sudo. Developer workstation / shared host → user mode.

Quick start

```
# System mode
sudo ./install.sh
sudo systemctl status abmeter
sudo journalctl -u abmeter -f

# User mode
./install.sh --user
systemctl --user status abmeter
journalctl --user -u abmeter -f
```

Single-instance prevention

Three independent layers ensure only one instance ever runs:

1. **systemd Type=simple** — systemd tracks the PID and never launches a second `ExecStart` while the first is alive.
2. **flock --nonblock** in `abmeter_wrapper.sh` — the wrapper takes an exclusive lock on `/run/abmeter/abmeter.lock`; if held, it exits with code 1.
3. **PID file** at `/run/abmeter/abmeter.pid` — written by the wrapper, removed on exit via a `trap`.

Configuration (wrapper)

Polling parameters live in `abmeter_wrapper.sh`:

Variable	Default	Description
POLL_COUNT	0	Iterations (0 = infinite)
POLL_INTERVAL	30	Seconds between polls
RAM_PCT_LIMIT	70	Flush+clear <code>TIME_SERIES_STORE</code> at this RAM % (ignored when <code>ENABLE_RAM_FLUSH=0</code>)
INSTALL_DIR	<code>/opt/abmeter</code>	Root of the installation

IP lists and output formats are configured inside `ab_power_meter_monitor.py` (the `IP_LIST` and `ENABLE_*` constants). After changing the wrapper, reload: `sudo systemctl restart abmeter`.

Installer flags

`install.sh` handles both services and both env strategies:

Flag	Effect
<code>--user</code>	Install to <code>~/.config/systemd/user</code> (no sudo)
<code>--exporter</code>	Also install the exporter service
<code>--only-exporter</code>	Install only the exporter (see Mode B)
<code>--python PATH</code>	Use an existing interpreter (e.g. a mamba env) instead of building a venv
<code>--mamba</code>	Create a mamba env (interpreter) + pip-install requirements
<code>--py37</code>	Use <code>requirements-py37.txt</code> (Python 3.7.x)
<code>--ips</code> , <code>--port</code> , <code>--interval</code> , <code>--max-history</code>	Exporter parameters

Example — wire the systemd service to a pure-conda env you built with `install-mamba.sh`:

```
./install-mamba.sh                                # creates env "abmeter"
sudo ./install.sh --exporter \
  --python "$(mamba run -n abmeter which python)"
```

Mode B — Prometheus exporter

`ab_prometheus_exporter.py` uses the monitor as a **library**: a background thread polls on a fixed cadence and updates `TIME_SERIES_STORE`; a custom collector reads the latest values on each scrape (no network I/O on scrape, so it can never trip `scrape_timeout`). The main thread blocks to keep the daemon HTTP server — and the process — alive.

This is why `ENABLE_SIGNAL_HANDLERS` exists: the exporter never calls `abm.main()`, and the monitor must not seize the process’s `SIGTERM/SIGINT`. The exporter is unaffected by `ENABLE_RAM_FLUSH` (it never calls the headless loop); it bounds memory with `--max-history` instead.

Run

```
pip install prometheus_client                    # plus core deps
python ab_prometheus_exporter.py \
  --ips 10.16.130.50,10.16.130.54 --interval 30 # serves :9184/metrics
```

Or install it as a service in one step:

```
sudo ./install.sh --only-exporter --ips 10.16.130.50,10.16.130.54 --port 9184
```

Flag	Default	Meaning
<code>--port</code>	9184	<code>/metrics</code> listen port
<code>--interval</code>	30	Seconds between background poll cycles
<code>--ips</code>	monitor <code>IP_LIST</code>	Comma-separated device IPs
<code>--retries</code>	2	Per-page fetch attempts
<code>--retry-delay</code>	2.0	Seconds between fetch retries

--max-history	3	Trailing samples retained per column (0 = unbounded)
---------------	---	--

Metrics exposed

ab_meter_value{ip,table,parameter,unit}	latest numeric value
ab_meter_up{ip}	1 if device responded on last poll
ab_meter_samples_total{ip,table}	accumulated sample count
ab_meter_poll_duration_seconds	wall-time of last poll cycle
ab_meter_last_poll_timestamp_seconds	unix time of last store update
ab_meter_polls_total	counter: poll cycles attempted
ab_meter_poll_errors_total	counter: wholesale poll failures
ab_meter_build_info{version,monitor_version}	constant 1

Parameter names (which contain spaces) are carried as **label values**, not metric names, so exposition is always valid.

Prometheus scrape config

```
scrape_configs:
- job_name: ab_power_meter
  static_configs:
  - targets: ["exporter-host:9184"]
```

Sample systemd unit (exporter)

```
# /etc/systemd/system/ab-exporter.service
[Unit]
Description=AB Power Meter Prometheus exporter
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=abmeter
WorkingDirectory=/opt/abmeter
ExecStart=/opt/abmeter/venv/bin/python /opt/abmeter/ab_prometheus_exporter.py \
--ips 10.16.130.50,10.16.130.54 --interval 30 --port 9184
Restart=on-failure
RestartSec=15

[Install]
WantedBy=multi-user.target
```

systemctl stop ab-exporter sends SIGTERM, which the exporter handles cleanly (it stops the poll loop and exits).

Mode C — driving the monitor from Python (the caller)

ab_meter_caller.py owns an outer loop and calls the monitor one poll at a time. run() returns abm.TIME_SERIES_STORE.

```
import ab_meter_caller as abc

data = abc.run(count=3, interval=30) # 3 samples
data = abc.run(count=0, interval=30) # until ab_stop.py
data = abc.run(count=0, interval=30, ram_pct=80) # higher flush threshold
data = abc.run(count=0, interval=30, enable_ram_flush=False) # never flush/clear

pwr = data["10.16.130.50"]["Real Time Power Table"]
pwr["columns"]["Total Real Power"] # [123.4, 124.1, 123.9]
pwr["timestamps_local"] # ["2026-07-22 12:00:00", ...]
```

CLI equivalents:

```
python ab_meter_caller.py --count 0 --interval 30
python ab_meter_caller.py --count 0 --interval 30 --no-ram-flush
python ab_meter_caller.py --count 10 --interval 60 --ram-pct 80
```

Parameter (column) names are the exact labels the meter reports. If a lookup returns nothing, print list(pwr["columns"].keys()) for that device.

Graceful stop

The headless service responds to two stop mechanisms:

```
# 1. systemd stop – sends SIGTERM; Python catches it and exits cleanly.
sudo systemctl stop abmeter

# 2. ab_stop.py – writes a STOP_COLLECTION sentinel the loop checks every 0.5 s.
/opt/abmeter/venv/bin/python3 /opt/abmeter/ab_stop.py
```

Both paths write all enabled output files before exiting. The `STOP_COLLECTION` file is also honoured by the Prometheus exporter’s poll loop, so `ab_stop.py` stops that too.

Note on `ENABLE_SIGNAL_HANDLERS = 0` : with the monitor’s own signal handlers disabled (the setting used when embedding, e.g. the exporter), the graceful-stop path is the `STOP_COLLECTION` file, not Ctrl-C — the host process owns the signals.

Disabling RAM dumps (flush / clear)

The monitor’s headless loop can, when system RAM crosses `MEM_RAM_PCT_LIMIT` (or the store/free-RAM thresholds), **flush** enabled file outputs and then **clear** the in-memory `TIME_SERIES_STORE` to reclaim memory. That clear is the “RAM dump”. Turning it off gives one uninterrupted, fully-accumulated store — at the cost of unbounded growth on long runs, so watch memory.

How you turn it off depends on the path:

Path	Do RAM dumps happen?	How to disable
Caller (<code>ab_meter_caller.run</code>)	Yes — it drives the headless loop	<code>run(..., enable_ram_flush=False)</code> or CLI <code>--no-ram-flush</code> (sets <code>abm.ENABLE_RAM_FLUSH=0</code>)
Headless service / direct	Yes	Set <code>ENABLE_RAM_FLUSH = 0</code> in the file, or launch with <code>--no-ram-flush</code>
Prometheus exporter (<code>ab_prometheus_exporter.py</code>)	No — never (it never runs the headless loop)	Nothing to disable; bound memory with <code>--max-history</code> instead (<code>0</code> = keep full history)

Worked example — `ab_ramflush_off_Example.py` demonstrates both the caller and a caller-driven Prometheus exporter with dumps disabled:

```
# Caller from Python – RAM dumps OFF (accumulates the full run)
python ab_ramflush_off_Example.py caller --count 0 --interval 30

# Caller-driven Prometheus exporter – RAM dumps OFF, memory bounded by trim
python ab_ramflush_off_Example.py prometheus --port 9184 --interval 30 --max-history 3

# Dedicated exporter – never dumps; keep full history with --max-history 0
python ab_prometheus_exporter.py --ips 10.16.130.50,10.16.130.54 --max-history 0
```

Trade-off: with dumps off, `TIME_SERIES_STORE` grows for the life of the run. The caller path accumulates unbounded (that’s the point — one complete series); the exporter paths use `--max-history` to bound memory *without* dumping, which is what you usually want for a long-lived `/metrics` endpoint.

Python 3.7.11 compatibility

The code is compatible with Python **3.7.11 through 3.12+** with no source changes — verified: no walrus operator, no self-documenting f-strings, no `BooleanOptionalAction`, no builtin-generic annotations, no `match`, no post-3.7 stdlib APIs, and numpy usage limited to `np.array` / `np.float64` (stable across numpy 1.21 → 2.x). Each runnable script carries an advisory `MIN_PYTHON = (3, 7, 11)` check that warns on an older interpreter but never aborts.

Only dependency versions differ. On Python 3.7.x the newest releases won’t install, so pin to the last 3.7-capable versions (all verified as installable cp37 wheels from PyPI):

Package	3.8+ (<code>requirements.txt</code>)	3.7.x (<code>requirements-py37.txt</code>)	Needed for
requests	≥ 2.31	2.31.0	core
beautifulsoup4	≥ 4.12	4.15.0	core
lxml	≥ 4.9	5.4.0	core
psutil	≥ 5.9	7.2.2	RAM-flush thresholds
prometheus_client	≥ 0.19	0.17.1	exporter (0.18+ drops 3.7)
numpy	≥ 1.24	1.21.6	FITS
astropy	≥ 5.2	4.3.1	FITS
openpyxl	≥ 3.1	3.1.3	XLSX
PySide6	≥ 6.6	6.5.3	GUI
matplotlib	≥ 3.7	3.5.3	GUI preview

conda-forge has largely dropped Python 3.7 builds; use the mamba env file (which installs via pip) or a pyenv-built 3.7.11 + venv. If `environment-py37.yml` cannot resolve `python=3.7.11` exactly, relax it to `python=3.7`.

Restart behaviour & log access

```
Restart=on-failure      → auto-restart on crash or non-zero exit
RestartSec=15           → wait 15 s before restarting
StartLimitBurst=5       → give up after 5 failures in 2 minutes
StartLimitIntervalSec=120
```

A clean exit (code 0 from `ab_stop.py`) does **not** trigger a restart.

```
sudo journalctl -u abmeter -f           # live tail (system mode)
sudo journalctl -u abmeter -n 100 --no-pager # last 100 lines
ls /opt/abmeter/ab_meter_output/logs/    # on-disk logs
journalctl --user -u abmeter -f         # user mode
```

Uninstall

```
# System mode
sudo systemctl stop abmeter
sudo systemctl disable abmeter
sudo rm /etc/systemd/system/abmeter.service
sudo rm /etc/logrotate.d/abmeter
sudo systemctl daemon-reload
sudo userdel abmeter          # optional: remove service account
sudo rm -rf /opt/abmeter      # optional: remove all data

# User mode
systemctl --user stop abmeter
systemctl --user disable abmeter
rm ~/.config/systemd/user/abmeter.service
systemctl --user daemon-reload
loginctl disable-linger "${id -un}" # if you enabled linger
rm -rf ~/.local/share/abmeter      # optional: remove all data
```

Troubleshooting

Symptom	Check
Service fails to start	<code>journalctl -u abmeter -n 50</code> — look for Python import errors or a missing venv
<code>flock</code> : another instance running	<code>cat /run/abmeter/abmeter.pid</code> — stale PID file; delete it and restart
No output files written	Verify <code>ENABLE_CSV</code> , <code>ENABLE_FITS</code> , ... in <code>ab_power_meter_monitor.py</code>
HTTP poll failures	Check <code>IP_LIST</code> ; verify network reachability from the service user
Service restart loop	<code>StartLimitBurst</code> hit → <code>sudo systemctl reset-failed abmeter && sudo systemctl start abmeter</code>
Exporter shuts down on <code>docker stop</code> / <code>systemctl reload</code>	Ensure <code>ENABLE_SIGNAL_HANDLERS = 0</code> (the caller sets it; direct <code>import abm</code> must set it) and that the poll runs with <code>count=0</code> so the main thread stays blocked
Store grows unbounded / high RAM	Headless: <code>ENABLE_RAM_FLUSH=1</code> (default) with a sane <code>MEM_RAM_PCT_LIMIT</code> . Exporter: lower <code>--max-history</code>
Prometheus target flapping (scrape timeout)	Confirm you're using the exporter (scrapes read the store, no I/O) — do not poll inside the scrape path
<code>prometheus_client</code> install fails on 3.7	Pin <code>prometheus_client==0.17.1</code> (0.18+ requires 3.8+)

Change log

Component	Version	Change
monitor	1.5.1	Added <code>ENABLE_SIGNAL_HANDLERS</code> ; guarded the signal-handler install so embedding hosts (exporter) aren't torn down by <code>SIGTERM</code> .
monitor	1.5.2	Added <code>ENABLE_RAM_FLUSH</code> master switch gating all automatic RAM/size flushes (headless + GUI).
monitor	1.5.3	Docs: requirements files, Python 3.7.11 support note + advisory version check.
monitor	1.5.4	Added import-safe CLI front-end (<code>--headless</code> , <code>--count</code> , <code>--interval</code> , <code>--ips</code> , <code>--ram-pct</code> , <code>--no-ram-flush</code>) so the service can launch it directly.
monitor	1.5.5	Robustness: restore <code>sys.stdout/stderr</code> even if the headless loop raises a <code>KeyboardInterrupt</code> exception in silent mode (previously left redirected to <code>/dev/null</code>).
kit	1.0.0	Added <code>install.sh</code> , <code>abmeter_wrapper.sh</code> ,

		abmeter.service, ab-exporter.service, abmeter_logrotate.conf, install-mamba.sh, environment-conda.yml.
example	1.0.1	Added ab_ramflush_off_Example.py (caller + Prometheus, RAM dumps off; quiets the monitor's per-poll config logging via a persistent filter).
caller	1.2.8	Sets ENABLE_SIGNAL_HANDLERS = 0 on import (exporter-safe).
caller	1.2.9	run(enable_ram_flush=...) kwarg and --no-ram-flush CLI flag.
caller	1.2.10	Docs / 3.7.11 note.
exporter	0.0.1	Initial reference Prometheus exporter.
exporter	0.0.3	Docs: ENABLE_RAM_FLUSH N/A note, 3.7.11 note, requirements reference.
example	1.1.2	Documents enable_ram_flush and the new setup.